

SPRINT PLANNING DOCUMENT

IntelliDocs AI Sprinting Analysis

Practical 6-Sprint Development Plan for the Requirement Analysis & Database Design

Table of Contents

1. Sprint Overview.....	2
2. Sprint 1: Foundation & Authentication	3
3. Sprint 2: Document Upload & Management	3
4. Sprint 3: Intelligent Document Processing.....	4
5. Sprint 4: RAG & Semantic Search	5
6. Sprint 5: RAG Chat & Advanced AI	5
7. Sprint 6: Dashboard, Security & Deployment	6
8. Overall Sprint Flow	7
9. Priority Classification.....	7
10. Future Enhancements	8
11. Recommended MVP Demo Path.....	8

1. Sprint Overview

This plan translates the IntelliDocs AI Requirement Analysis and its accompanying database table structures into a practical, buildable roadmap. The requirements span authentication, document upload and processing, OCR, classification, metadata extraction, summarization, embeddings and vector search, retrieval-augmented generation (RAG), chat history, document comparison, dashboarding, role-based access control, and error handling.

Sprint	Focus	Main Deliverables
Sprint 1	Project & Database Foundation	Architecture, SQL Server DB, authentication
Sprint 2	Document Management	Upload, storage, validation, document dashboard
Sprint 3	AI Document Processing	OCR, text extraction, classification, entities, summarization
Sprint 4	RAG & Semantic Search	Chunking, embeddings, vector search, RAG
Sprint 5	AI Chat & Advanced Features	Multi-document QA, citations, comparison, chat history
Sprint 6	Security, Dashboard & Deployment	RBAC, analytics, error handling, testing, Azure deployment

2. Sprint 1: Foundation & Authentication

Goal: Establish the application architecture and secure user management.

Backend

- Create ASP.NET Core Web API
- Configure layered architecture
- Configure Entity Framework Core
- Connect SQL Server
- Implement Repository / Service patterns
- Configure JWT authentication
- Implement password hashing

Database

Create initial tables: Roles, Users, DocumentCategories.

The database design already defines Users with JWT-compatible authentication data such as email, password hash, role, and account status.

Frontend

- React project setup
- Tailwind CSS
- Login page
- Registration page
- Basic routing

Sprint 1 Output: A user can register, log in, receive a JWT, and access protected pages.

3. Sprint 2: Document Upload & Management

Goal: Allow authenticated users to securely upload and manage documents.

Features

- PDF upload
- PNG / JPG / JPEG upload

- DOCX upload
- File type validation
- File size validation
- Secure storage
- Processing status tracking
- Document listing, search / filter
- Download
- Delete / soft delete

These correspond mainly to FR-03, FR-04, and FR-17.

Database

Implement Documents, with relationships to Users and DocumentCategories.

The Documents table already contains ownership, file type, size, storage path, category, and processing status fields.

Frontend

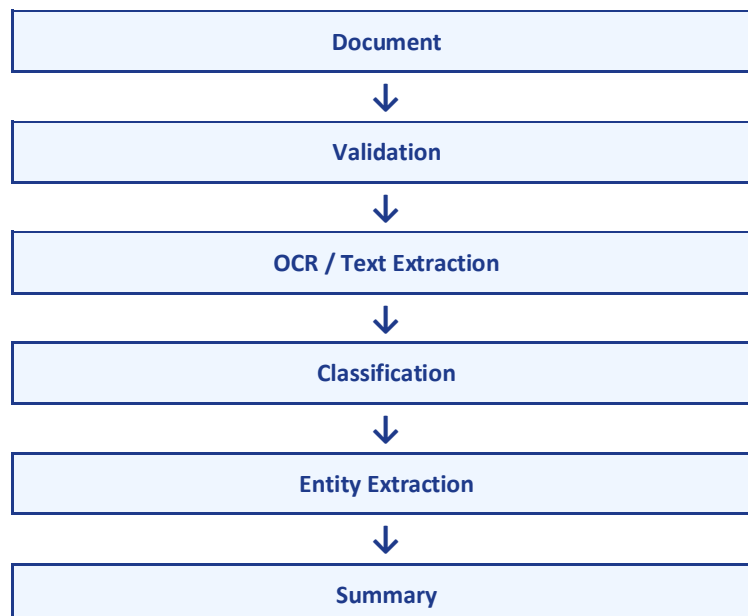
- Upload component
- Document library
- Document cards / table
- Processing-status indicators
- Delete / download actions

Sprint 2 Output: User can upload and manage documents through the UI.

4. Sprint 3: Intelligent Document Processing

Goal: Convert uploaded documents into structured, searchable information.

Processing Pipeline



The requirement analysis explicitly defines OCR/text extraction, classification, information extraction, and summarization as core functional requirements.

Implement

- OCR using Azure AI Document Intelligence

- Digital PDF text extraction
- Document classification: Invoice, Contract, Resume, Report, Legal document
- Entity extraction: Names, Dates, Amounts, Organizations, Document numbers
- AI summarization
- Processing status updates

Database

Use DocumentText, ExtractedEntities, DocumentSummaries.

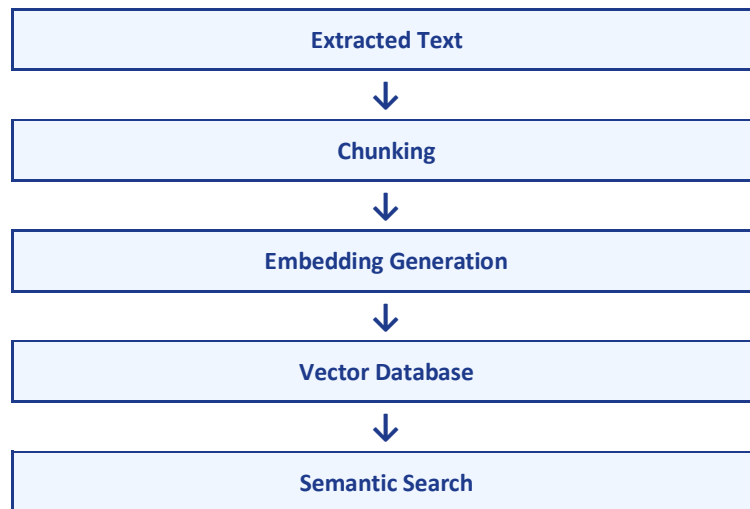
These tables are specifically structured for extracted text, entity values/confidence, and AI-generated summaries.

Sprint 3 Output: Uploading a document automatically produces extracted text, document type, important entities, and an AI summary.

5. Sprint 4: RAG & Semantic Search

Goal: Build the core AI knowledge-retrieval system. This is the most technically important sprint.

Pipeline



The requirement analysis specifically defines document chunking, embedding generation, and semantic search as FR-09, FR-10, and FR-11.

Implement

- Logical text chunking
- Preserve page numbers
- Generate embeddings
- Store vector references
- Implement similarity search
- Build retrieval API
- Optimize chunk size / top-k retrieval

Database

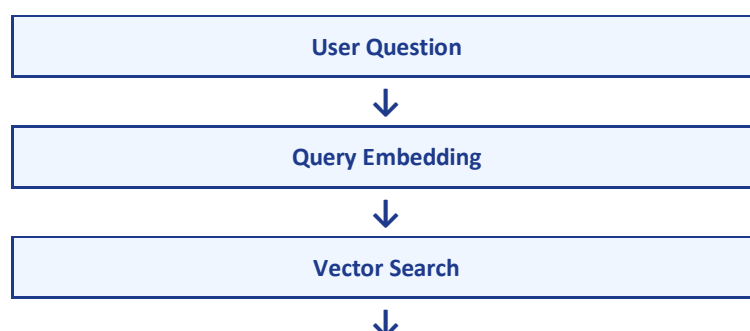
Implement DocumentChunks: stores chunk order, page number, chunk text, and vector database reference.

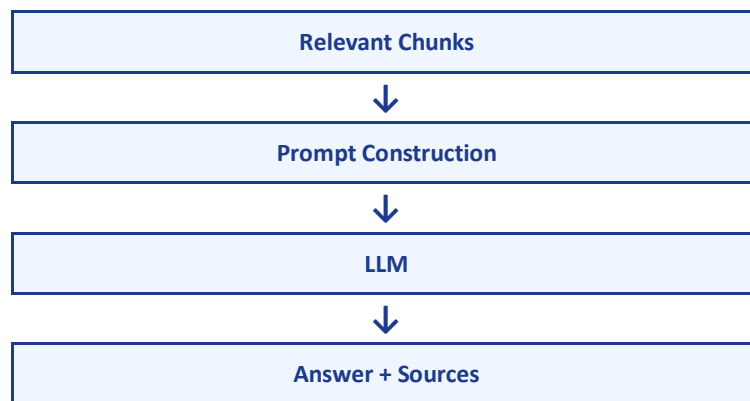
Sprint 4 Output: A user can type a natural-language query, and the system retrieves the most relevant document chunks.

6. Sprint 5: RAG Chat & Advanced AI

Goal: Turn semantic retrieval into a complete conversational AI experience.

RAG Pipeline





This flow is directly specified in the requirement document.

Implement

- RAG question answering
- Conversational chat UI
- Multi-document questions
- Source citations, page / section references
- Chat history: continue or delete previous conversations
- Document comparison

FR-12 through FR-16 cover these capabilities.

Database

Implement Conversations, ConversationDocuments, ChatMessages, AnswerSources, DocumentComparisons.

The database design already maps these entities to conversations, messages, source chunks/pages, and document comparisons.

Sprint 5 Output: A user can have a conversation such as:

User: Compare the payment terms in Contract A and Contract B.

AI: Contract A requires payment within 30 days, while Contract B specifies 45 days.

Sources: Contract A: Page 4, Contract B: Page 6

7. Sprint 6: Dashboard, Security & Deployment

Goal: Make the system production-ready.

Security

- JWT authentication
- Role-based authorization
- Secure file access
- Input validation
- API authorization
- HTTPS
- Prevent unauthorized document access

These security requirements are explicitly listed in the SRS.

Dashboard

- Total documents
- Processing status
- Category breakdown
- Storage usage
- AI query statistics

Error Handling

- Upload errors
- OCR errors
- AI API errors
- Database errors

The database also includes SystemErrors for recording OCR, AI, upload, and database failures.

Testing

- Unit testing
- API testing
- Integration testing
- Authentication testing
- RAG accuracy testing
- File-upload testing
- Security testing

Deployment

The specified technology stack includes React, ASP.NET Core, SQL Server, Azure AI Document Intelligence, Azure OpenAI / compatible LLM, Azure AI Search / vector database, GitHub, and Microsoft Azure.

Sprint 6 Output: A secure, tested, and deployable IntelliDocs AI platform.

8. Overall Sprint Flow



9. Priority Classification

MUST HAVE: MVP

- Registration / Login
- Document upload & storage
- OCR / text extraction
- Classification
- Entity extraction
- Summarization
- Chunking, embeddings, semantic search
- RAG Q&A with source citations
- Document management

SHOULD HAVE

- Multi-document Q&A
- Chat history
- Document comparison
- Dashboard
- RBAC
- Advanced error logging

10. Future Enhancements

Beyond the MVP scope, the requirement analysis lists the following future enhancements: voice-based document queries, multilingual document processing, handwriting recognition, automatic email/document ingestion, advanced fraud/anomaly detection, document version tracking, team-based document sharing, mobile applications, and advanced analytics.

11. Recommended MVP Demo Path

If the goal is to actually build and demonstrate this project, the strongest first demo path is:

**Login → Upload PDF → OCR → Extract Text → Generate Summary → Ask Question → RAG Answer
→ Show Page Citation**

This gives the strongest demonstration of the project's **.NET + AI + RAG** capabilities without needing every feature completed.